

Teoría de la Computación

Dr. Javier Baladrón

Complejidad

Sea M una máquina de Turing determinista que se detiene para todas las entradas. La complejidad temporal, de M es la función

$$f : \mathbb{N} \rightarrow \mathbb{N},$$

donde $f(n)$ es el número máximo de pasos que M utiliza sobre cualquier entrada de longitud n .

Complejidad

Recordemos de Diseño de Algoritmos:

Sean f y g funciones $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$. Decimos que

$$f(n) = O(g(n))$$

si existen enteros positivos c y n_0 tales que para todo entero

$$n \geq n_0,$$

se cumple que

$$f(n) \leq c g(n).$$

Complejidad

Analicemos una maquina que decide $A = \{0^k 1^k | k \geq 0\}$.

M_1 = Sobre la entrada w :

1. Recorrer la cinta y rechazar si se encuentra un 0 a la derecha de un 1.
2. Repetir mientras queden 0s y 1s en la cinta:
 - 2.1 Recorrer la cinta tachando un único 0 y un único 1.
3. Si aún quedan 0s después de que todos los 1s hayan sido tachados, o si aún quedan 1s después de que todos los 0s hayan sido tachados, rechazar. En caso contrario, si no quedan ni 0s ni 1s en la cinta, aceptar.

Etapas 1: $O(n)$

Etapas 2: Cada recorrido es $O(n)$, se hacen maximo $n/2$ recorridos, $(n/2)O(n) = O(n^2)$.

Etapas 3: $= O(n)$

De esta forma M_1 es de $O(n^2)$.

Complejidad

Sea $t : \mathbb{N} \rightarrow \mathbb{R}^+$ una función. Definimos la clase de complejidad temporal $\text{TIME}(t(n))$, como la colección de todos los lenguajes que son decidibles por una máquina de Turing que opera en tiempo $O(t(n))$.

$$A = \{0^k 1^k \mid k \geq 0\} \in \text{TIME}(n^2)$$

Complejidad

M_2 = Sobre la entrada w :

1. Recorrer la cinta y rechazar si se encuentra un 0 a la derecha de un 1.
2. Repetir mientras queden algunos 0s y algunos 1s en la cinta:
 - 2.1 Recorrer la cinta verificando si el número total de 0s y 1s restantes es par o impar. Si es impar, rechazar.
 - 2.2 Recorrer nuevamente la cinta, tachando uno de cada dos 0s comenzando con el primer 0, y luego tachando uno de cada dos 1s comenzando con el primer 1.
 - 2.3 Si no quedan ni 0s ni 1s en la cinta, aceptar. En caso contrario, rechazar.

Eta 1: $O(n)$

La Eta 2.2 elimina la mitad de los 0s y 1s en cada iteracion, máximo $\log_2 n$.

La Eta 2.2 se repite n veces.

M_2 es de $O(n \log n)$

Complejidad

M_3 = Sobre la entrada w :

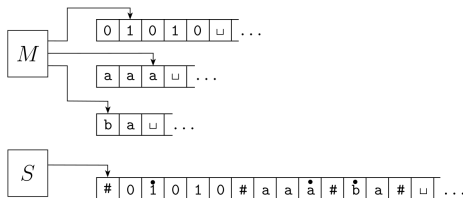
1. Recorrer la cinta 1 y rechazar si se encuentra un 0 a la derecha de un 1.
2. Recorrer los 0s en la cinta 1 hasta el primer 1. Al mismo tiempo, copiar los 0s en la cinta 2.
3. Recorrer los 1s en la cinta 1 hasta el final de la entrada. Por cada 1 leído en la cinta 1, tachar un 0 en la cinta 2. Si todos los 0s son tachados antes de que todos los 1s hayan sido leídos, rechazar.
4. Si todos los 0s han sido tachados, aceptar. Si queda algún 0, rechazar.

Si la máquina tiene 2 cintas podemos decidir en $O(n)$.

Complejidad

Sea $t(n)$ una función tal que $t(n) \geq n$. Entonces, toda máquina de Turing multicinta que opera en tiempo $t(n)$ tiene una máquina de Turing equivalente de una sola cinta que opera en tiempo $O(t^2(n))$.

Sabemos de clases anteriores que toda máquina multicinta tiene una máquina de una sola cinta equivalente.



Complejidad

La máquina con una sola cinta debe ejecutar un paso de cada cinta a la vez.

Debe recorrer la representación de cada cinta para encontrar donde esta el cabezal.

El largo máximo de cada cinta es $t(n)$.

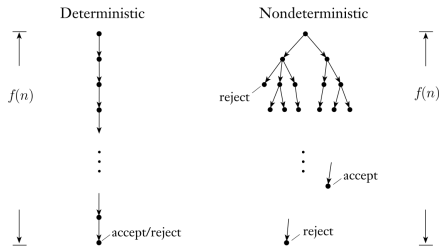
Si ejecuta $t(n)$ pasos y por cada paso busca el cabezal en $O(t(n))$ tenemos $O(t^2(n))$

Complejidad

Sea M una máquina de Turing no determinista que decide algún lenguaje. El tiempo de ejecución de M es la función

$$f : \mathbb{N} \rightarrow \mathbb{N},$$

donde $f(n)$ es el número máximo de pasos que M utiliza en cualquier rama de su computación para cualquier entrada de longitud n .



Complejidad

Sea $t(n)$ una función tal que $t(n) \geq n$. Entonces, toda máquina de Turing no determinista de una sola cinta que opera en tiempo $t(n)$ tiene una máquina de Turing determinista equivalente de una sola cinta que opera en tiempo $2^{O(t(n))}$.

Cada rama del árbol de computación $t(n)$.

Cada nodo del árbol puede tener como máximo b hijos.

El número total de hojas en el árbol es, como máximo, $b^{t(n)}$.

La simulación visita todos los nodos de profundidad d antes de continuar con los de profundidad $d+1$.

El número total de nodos es menor que el doble del número máximo de hojas, por lo que podemos acotarlo por $O(b^{t(n)})$.

El tiempo de ejecución de D es $O(t(n)b^{t(n)}) = 2^{O(t(n))}$.

$$b^{t(n)} = 2^{t(n) \log_2 b} = 2^{O(t(n))}.$$

Complejidad

P es la clase de lenguajes que pueden decidirse en tiempo polinomial por una máquina de Turing determinista de una sola cinta. En otras palabras,

$$P = \bigcup_{k \geq 1} \text{TIME}(n^k).$$

1. La clase P es invariante para todos los modelos de computación que son polinomialmente equivalentes a la máquina de Turing determinista de una sola cinta.
2. La clase P corresponde aproximadamente al conjunto de problemas que pueden resolverse de manera realista en un computador.

Complejidad

Un **verificador** para un lenguaje A es un algoritmo V , tal que

$$A = \{ w \mid V \text{ acepta } \langle w, c \rangle \text{ para alguna cadena } c \}.$$

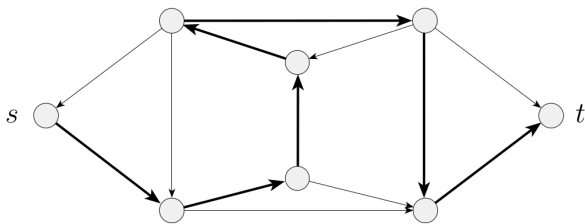
Medimos el tiempo de ejecución de un verificador únicamente en función de la longitud de w . Por lo tanto, un verificador de tiempo polinomial se ejecuta en tiempo polinomial respecto de la longitud de w .

Un lenguaje A es **polinomialmente verificable** si posee un verificador de tiempo polinomial.

c es llamado el **certificado** o la prueba.

Complejidad

$\text{HAMPATH} = \{ \langle G, s, t \rangle \mid G \text{ es un grafo dirigido que contiene un camino hamiltoniano desde } s \text{ hasta } t \}$.



Verificar la existencia de un camino hamiltoniano puede ser mucho más fácil que determinar su existencia.

Consideremos $\overline{\text{HAMPATH}}$, no conocemos una forma de verificar dicha inexistencia sin utilizar el algoritmo exponencial.

Complejidad

La clase **NP** es el conjunto de lenguajes que poseen verificadores de tiempo polinomial.

La clase **NP** es el conjunto de lenguajes decididos por una MT no determinista en tiempo polinomial.

Complejidad

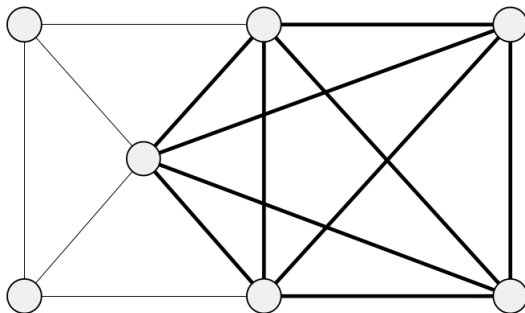
La siguiente es una máquina de Turing no determinista (NTM) que decide el problema HAMPATH en tiempo polinomial no determinista.

N_1 = Sobre la entrada $\langle G, s, t \rangle$, donde G es un grafo dirigido con nodos s y t :

1. Escriba una lista de m números, $p_1, \dots, p_m$ donde m es el número de nodos de G . Cada número de la lista se selecciona de manera no determinista entre 1 y m .
2. Verifique si existen repeticiones en la lista. Si encuentra alguna, rechace.
3. Verifique que $s = p_1$ y que $t = p_m$. Si alguna de estas condiciones falla, rechace.
4. Para cada i entre 1 y $m-1$, verifique si (p_i, p_{i+1}) es una arista de G . Si alguna no lo es, rechace. En caso contrario, todas las verificaciones han sido superadas, por lo que acepte.

Complejidad

$\text{CLIQUE} = \{ \langle G, k \rangle \mid G \text{ es un grafo no dirigido que contiene un } k\text{-clique} \}.$



Complejidad

CLIQUE $\in$ NP.

La clique es el certificado c .

$V =$ Sobre la entrada $\langle \langle G, k \rangle, c \rangle$:

1. Verifique si c es un subgrafo de G con k nodos.
2. Verifique si G contiene todas las aristas que conectan los nodos de c .
3. Si ambas verificaciones son satisfactorias, acepte; en caso contrario, rechace.

Complejidad

$N =$ Sobre la entrada $\langle G, k \rangle$, donde G es un grafo:

1. Seleccione de manera no determinista un subconjunto c de k nodos de G .
2. Verifique si G contiene todas las aristas que conectan los nodos de c .
3. Si la respuesta es afirmativa, acepte; en caso contrario, rechace.

Complejidad

$SUBSET-SUM = \{ \langle S, t \rangle \mid S = \{x_1, \dots, x_k\} \text{ y para algún } \{y_1, \dots, y_l\} \subset \{x_1, \dots, x_k\} \text{ tenemos } \sum y_i = t \}$

Por ejemplo $\langle \{4, 11, 16, 21, 27\}, 25 \rangle \in SUBSET - SUM$.

$SUBSET - SUM \in NP$.

Complejidad

El subconjunto es el certificado.

$V =$ Sobre la entrada $\langle \langle S, t \rangle, c \rangle$:

1. Verifique si c es una colección de números cuya suma es t .
2. Verifique si S contiene todos los números que aparecen en c .
3. Si ambas verificaciones son satisfactorias, acepte; en caso contrario, rechace.

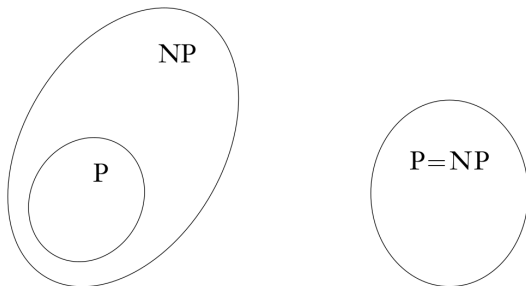
Complejidad

$N =$ Sobre la entrada $\langle S, t \rangle$:

1. Seleccione de manera no determinista un subconjunto c de los números de S .
2. Verifique si los números de c suman t .
3. Si la verificación es satisfactoria, acepte; en caso contrario, rechace.

Complejidad

¿ $P=NP$?



Complejidad

Recordemos:

Una función $f : \Sigma^* \rightarrow \Sigma^*$ es **computable en tiempo polinomial** si existe una máquina de Turing de tiempo polinomial M que, para cualquier entrada w , se detiene con $f(w)$ como único contenido de su cinta.

Sean A y B dos lenguajes. Decimos que A es reducible en tiempo polinomial a B , y escribimos $A \leq_P B$, si existe una función computable en tiempo polinomial $f : \Sigma^* \rightarrow \Sigma^*$ tal que

$$\forall w \in \Sigma^*, \quad w \in A \iff f(w) \in B.$$

La función f recibe el nombre de **reducción en tiempo polinomial** de A a B .

Complejidad

Si $A \leq_P B$ y $B \in P$, entonces $A \in P$.

Sea M el algoritmo de tiempo polinomial que decide B y sea f la reducción en tiempo polinomial de A a B .

$N =$ Sobre la entrada w :

1. Calcule $f(w)$.
2. Ejecute M con entrada $f(w)$ y produzca la misma salida que produzca M .

Complejidad

Una **fórmula booleana** está en **forma normal conjuntiva** (CNF) si está compuesta por varias cláusulas conectadas mediante operadores $\wedge$, como en

$$(x_1 \vee x_2 \vee x_3 \vee x_4) \wedge (x_3 \vee x_5 \vee x_6) \wedge (x_3 \vee x_6).$$

Una fórmula está en *3-CNF* si todas sus cláusulas contienen exactamente tres literales, como en

$$(x_1 \vee x_2 \vee x_3) \wedge (x_3 \vee x_5 \vee x_6) \wedge (x_3 \vee x_6 \vee x_4) \wedge (x_4 \vee x_5 \vee x_6).$$

Definimos

$$3SAT = \{\langle \varphi \rangle \mid \varphi \text{ es una fórmula 3-CNF satisfacible}\}.$$

Complejidad

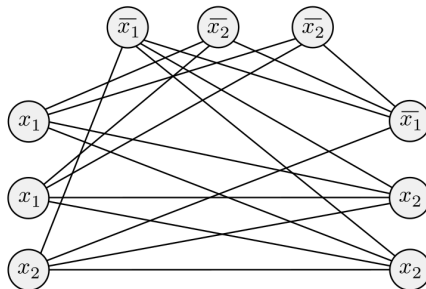
$3SAT \leq_P CLIQUE$.

Generamos un grafo G con:

Un nodo por cada literal (grupos de 3).

No hay arista entre nodos del mismo grupo.

No hay arista entre nodos contradictorios.



$$\phi = (x_1 \vee x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2 \vee x_2).$$

Complejidad

Un lenguaje B es **NP-completo** si satisface las siguientes dos condiciones:

1. $B \in NP$.
2. Todo lenguaje $A \in NP$ es reducible en tiempo polinomial a B .

Es decir,

$$\forall A \in NP, \quad A \leq_P B.$$

Complejidad

Si B es NP-completo y $B \in P$ entonces $P = NP$.

Si B es NP-completo y $B \leq_P C$ para C en NP, entonces C es NP-completo.

Complejidad

SAT es NP-completo.

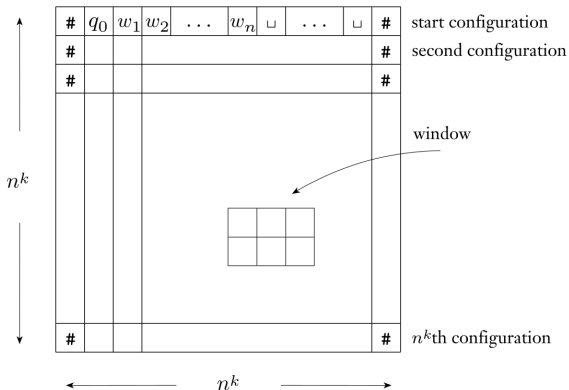
Primero demostrar que SAT es NP, TAREA.

A continuación demostraremos que cualquier problema $A \in NP$ se puede reducir en tiempo polinomial a SAT.

Complejidad

Sea N una máquina de Turing no determinista que decide A en tiempo n^k , para alguna constante k .

Un **tableau** para N sobre la entrada w es una tabla de dimensiones $n^k \times n^k$ cuyas filas corresponden a las configuraciones de una rama de la computación de N sobre la entrada w .



Complejidad

Dada una entrada w , la reducción construye una fórmula booleana φ .

Sea $C = Q \cup \Gamma \cup \{\#\}$. Para cada i y j entre 1 y n^k , y para cada símbolo $s \in C$, tenemos una variable $x_{i,j,s}$.

La fórmula φ es la conjunción de cuatro partes:

$$\varphi = \varphi_{\text{cell}} \wedge \varphi_{\text{start}} \wedge \varphi_{\text{move}} \wedge \varphi_{\text{accept}}.$$

φ_{cell} asegura que haya un símbolo por celda.

$$\varphi_{\text{cell}} = \bigwedge_{i,j} \left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{s,t \in C, s \neq t} (\neg x_{i,j,s} \vee \neg x_{i,j,t}) \right).$$

Complejidad

φ_{start} asegura la configuración inicial.

$$\begin{aligned}\varphi_{\text{start}} = & x_{1,1,\#} \wedge x_{1,2,q_0} \wedge x_{1,3,w_1} \wedge x_{1,4,w_2} \wedge \cdots \wedge x_{1,n+2,w_n} \\ & \wedge x_{1,n+3,\sqcup} \wedge \cdots \wedge x_{1,n^k-1,\sqcup} \wedge x_{1,n^k,\#}.\end{aligned}$$

La fórmula φ_{accept} asegura que aparezca una configuración de aceptación en el tableau.

$$\varphi_{\text{accept}} = \bigvee_{1 \leq i,j \leq n^k} x_{i,j,q_{\text{accept}}}.$$

Complejidad

La fórmula φ_{move} garantiza que cada fila del tableau corresponda a una configuración que sigue legalmente a la configuración de la fila anterior, de acuerdo con las reglas de transición de N .

Ejemplos validos:

(a)

a	q_1	b
q_2	a	c

(b)

a	q_1	b
a	a	q_2

(c)

a	a	q_1
a	a	b

(d)

#	b	a
#	b	a

(e)

a	b	a
a	b	q_2

(f)

b	b	b
c	b	b

Ejemplos no validos:

(a)

a	b	a
a	a	a

(b)

a	q_1	b
q_2	a	a

(c)

b	q_1	b
q_2	b	q_2

$$\delta(q_1, a) = \{(q_1, b, R)\} \text{ y } \delta(q_1, b) = \{(q_2, c, L), (q_2, a, R)\}$$

Complejidad

$$\varphi_{\text{move}} = \bigwedge_{\substack{1 \leq i < n^k \\ 1 < j < n^k}} (\text{la ventana } (i, j) \text{ es legal}).$$

Dado que el tableau tiene dimensiones $n^k \times n^k$, el número total de variables es $O(n^{2k})$.

La fórmula φ_{cell} contiene un fragmento de tamaño fijo para cada celda del tableau. Su tamaño es $O(n^{2k})$

La fórmula φ_{start} tiene un fragmento para cada celda de la fila superior, por lo que su tamaño es $O(n^k)$

Las fórmulas φ_{move} y φ_{accept} contienen cada una un fragmento de tamaño fijo para cada celda del tableau, por lo que su tamaño es $O(n^{2k})$

El **tamaño total** es entonces $O(n^{2k})$.

Complejidad

$3SAT = \{\langle \varphi \rangle \mid \varphi \text{ es una fórmula 3-CNF satisfacible}\}.$

3SAT es NP-completo.

Cualquier fórmula puede transformarse a CNF:

1. Utilizamos las leyes de De Morgan:

$$\neg(A \wedge B) = (A \vee \neg B) \text{ y } \neg(A \vee B) = (\neg A \wedge \neg B)$$

2. Podemos distribuir:

$$A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C)$$

$$(A \wedge B) \vee C = (A \vee C) \wedge (B \vee C)$$

Existen transformaciones más eficientes, por ejemplo transformación de Tseitin.

Complejidad

Podemos transformar cualquier formula en CNF a 3-CNF:

En cada clausula con más de 3 variables, dividimos en más clausulas.

$$(a_1 \vee a_2 \vee \dots \vee a_l) = \\ (a_1 \vee a_2 \vee z_1) \wedge (\neg z_1 \vee a_3 \vee z_2) \wedge (\neg z_2 \vee a_4 \vee z_3) \wedge \dots \wedge (\neg z_{l-3} \vee a_{l-1} \vee a_l)$$

Por ejemplo $(a_1 \vee a_2 \vee a_3 \vee a_4) = (a_1 \vee a_2 \vee z) \wedge (\neg z \vee a_3 \vee a_4)$

Por lo tanto podemos transformar la formula de la demostración anterior en 3-CNF.

3-CNF es entonces NP-completo.

Complejidad

Recordar: $3SAT \leq_P CLIQUE$.

Por lo tanto **CLIQUE es NP-completo**.

VERTEX-COVER =

$\{\langle G, k \rangle \mid G \text{ es un grafo no dirigido con un vertex-cover de } k \text{ nodos}\}$.

VERTEX-COVER es NP-Completo.

VERTEX-COVER es NP: el certificado es el vertex-cover.

3SAT es reducible polinomialmente a VERTEX-COVER.

VERTEX-COVER: es un subconjunto de vértices C tal que toda arista del grafo tiene al menos uno de sus extremos en C .

Complejidad

La reducción es un mapa entre una fórmula lógica y un grafo.

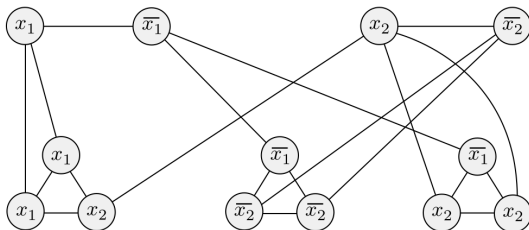
Por cada variable x se produce una arista entre nodos con etiqueta x y $\neg x$.

Se crean tres nodos por cada clausula que se conectan entre si.

Nodos con la misma etiqueta se conectan entre si.

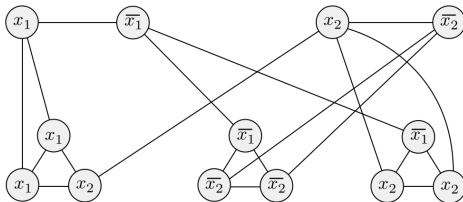
Si la fórmula es

$(x_1 \vee x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_2) \wedge (\neg x_1 \vee x_2 \vee x_2)$, el grafo es:



Complejidad

$$(x_1 \vee x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_2) \wedge (\neg x_1 \vee x_2 \vee x_2)$$



Partimos con una signacion verdadera.

Colocamos en el vertex-cover los nodos que corresponden a los literales verdaderos de la asignación.

Seleccionamos un literal verdadero en cada cláusula e incorporamos al vertex cover los otros dos nodos de cada gadget de cláusula.

Cualquier vertex cover debe contener un nodo en cada gadget de variable y dos nodos en cada gadget de cláusula para cubrir todas las aristas.

Complejidad

El problema del camino Hamiltoniano consiste en determinar si un grafo tiene un camino entre los nodos s y t que pase por todos los nodos exactamente una vez.

HAMPATH es NP-completo.

HAMPATH es NP: certificado es el camino.

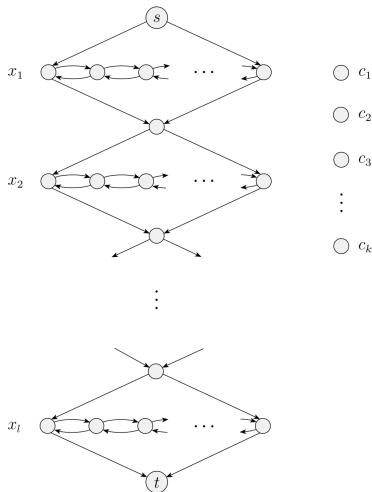
3SAT es polinomialmente reducible a HAMPATH.

Vamos a construir un grafo dirigido G con 2 nodos, s y t , donde un camino Hamiltoniano entre s y t existe sólo si la fórmula se satisface.

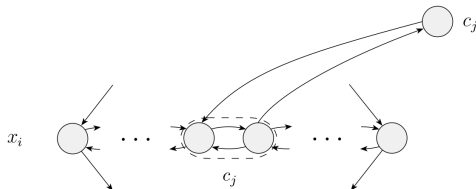
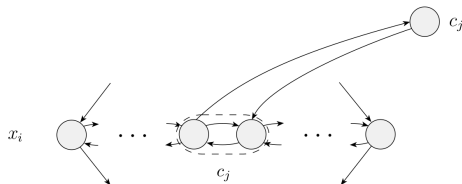
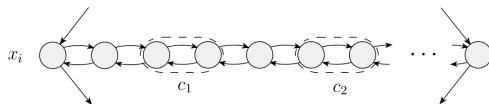
Complejidad

$$(a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \dots \wedge (a_k \vee b_k \vee c_k)$$

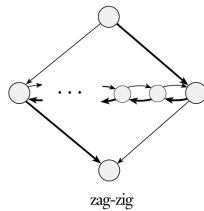
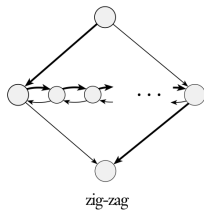
La estructura general del grafo:



Complejidad



Complejidad



Ejercicios

Demuestre que P es cerrado bajo la unión, concatenación y complemento.

Ejercicios

Demuestre que P es cerrado bajo la unión, concatenación y complemento.

Sean $A, B \in P$. Entonces existen máquinas deterministas M_A y M_B tales que:

M_A decide A en tiempo $p(n)$

M_B decide B en tiempo $q(n)$

Para la unión, diseñamos una máquina M :

1. Ejecutar M_A sobre w .
2. Si acepta, aceptar.
3. Ejecutar M_B sobre w .
4. Si acepta, aceptar.
5. En otro caso, rechazar.

El tiempo de ejecución es $p(|w|) + q(|w|)$, polinomial.

Ejercicios

Queremos demostrar que $A \circ B = \{xy | x \in A, y \in B\}$ esta en P.

Diseñamos una máquina M:

1. Para cada división $w=xy$:
 - 1.1 ejecutar M_A sobre x ,
 - 1.2 ejecutar M_B sobre y .
2. Si alguna división satisface que ambas máquinas aceptan, aceptar.
3. Si ninguna funciona, rechazar.

Hay $n+1$ divisiones. Cada prueba cuesta $p(n) + q(n)$. Tenemos entonces $(n+1)(p(n) + q(n))$, polinomial.

Ejercicios

Queremos demostrar que $\bar{A} \in P$.

Diseñamos una máquina M :

1. Ejecutar M_A sobre w .
2. Si M_A acepta, rechazar.
3. Si M_A rechaza, aceptar.

M realiza exactamente la misma computación que M_A , por lo que utiliza tiempo $p(n)$

Ejercicios

Un triángulo en un grafo no dirigido es un 3-clique, es decir, un conjunto de tres vértices tales que cada par de ellos está conectado por una arista.

Demuestre que $TRIANGLE \in P$, donde
 $TRIANGLE = \{ \langle G \rangle \mid G \text{ contiene un triángulo} \}.$

Ejercicios

Sea $G=(V,E)$ un grafo no dirigido con n vértices.

1. Enumerar todos los triples de vértices distintos (u,v,w) .
2. Para cada triple, verificar si las tres aristas $(u,v),(u,w),(v,w)$ pertenecen a E .
3. Si existe algún triple para el cual las tres aristas están presentes, aceptar.
4. Si se revisan todos los triples sin encontrar ninguno, rechazar.

El número de triples posibles es

$$\binom{n}{3} = \frac{n(n-1)(n-2)}{6} = O(n^3)$$

Ejercicios

Sea $DOUBLE - SAT =$

$\{ \langle \phi \rangle \mid \phi \text{ tiene al menos dos asignaciones satisfactorias} \}$.

Demuestre que DOUBLE-SAT es NP-completo.

Ejercicios

$DOUBLE - SAT \in NP$: Un certificado consiste en dos asignaciones distintas.

$DOUBLE-SAT$ es NP-hard: Construiremos una fórmula ψ tal que $\phi \in SAT \iff \psi \in DOUBLE - SAT..$

Entonces $\psi = \phi \wedge (y \vee \neg y)$. y es una variable nueva